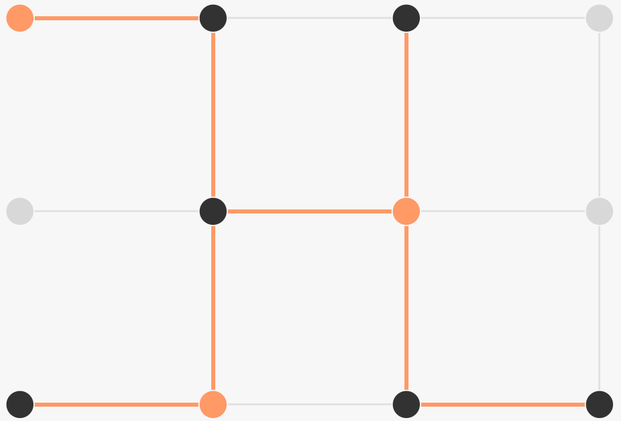
# Distributed HDBSCAN & DBSCAN

with Apache Spark



MSBD 5003 Deep Project

Manual implementation · Spark RDDs · Density clustering · Graph compression

# Presentation roadmap

Five parts, one through-line: variable density meets distributed graph computation.


### Introduction

Why DBSCAN struggles

### Algorithm

From eps to hierarchy

### Spark Implementation

Partition · Local MST · Merge

### Experiments + Discussion

Quality, scaling, limitations

### Conclusion

What worked and what bottlenecked

01

# Introduction

The project starts from a simple tension: density clustering is useful, but global thresholds are brittle.

# Problem background

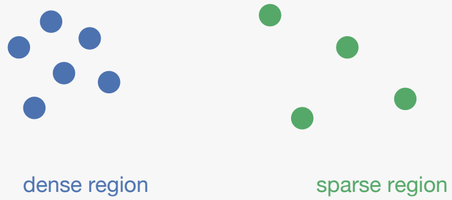
Density clustering is useful, but scaling it correctly is not trivial.

**We study density clustering when both density and data scale change.**

The algorithmic issue is variable density; the systems issue is how to distribute graph-heavy logic in Spark.

### 1 Density clustering goal

discover arbitrary shapes
and mark noise




### 2 Fixed eps breaks

one global radius cannot fit
both dense and sparse areas

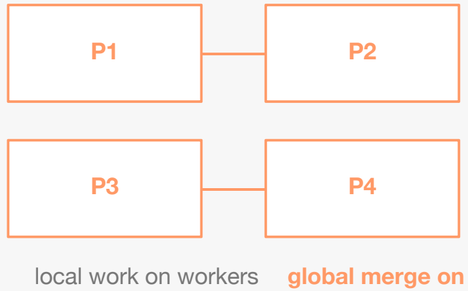
**small eps**
**misses sparse points**

**large eps**
**merges regions**

**HDBSCAN replaces one eps with a density hierarchy.**

### 3 Spark adds scale

neighbor and graph logic
cross partition boundaries




**Project focus: distributed DBSCAN baseline + HDBSCAN-inspired Spark pipeline + scalability analysis.**

# Project objective

Build the algorithmic internals, not a wrapper around a library.

**Distributed DBSCAN baseline** → **Distributed HDBSCAN-inspired pipeline** → **Experimental scalability analysis**

### Baseline

grid partitioning · ghost points · local DBSCAN · Union-Find merge

### Main pipeline

KD-tree partitioning · local MRD · Local MST compression · global MST

### Evaluation

cluster quality · strong scaling · data scalability · real taxi data

**No MLlib black box**

**Manual Union-Find + Kruskal**

**RDD transformations**

**Phase-level timing**

02

# Algorithm Overview

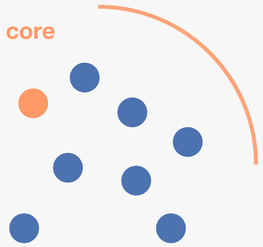
The algorithm story moves from fixed-radius density to graph-based density hierarchy.

# Distributed DBSCAN baseline

Local DBSCAN is simple; boundary correctness requires partition-aware merging.

## Inside each Spark partition

DBSCAN still uses the classic eps / minPts rule.




eps neighborhood

1 **Find neighbors**

2 **Classify core / border / noise**

3 **Expand density-connected cluster**

## Across Spark partitions

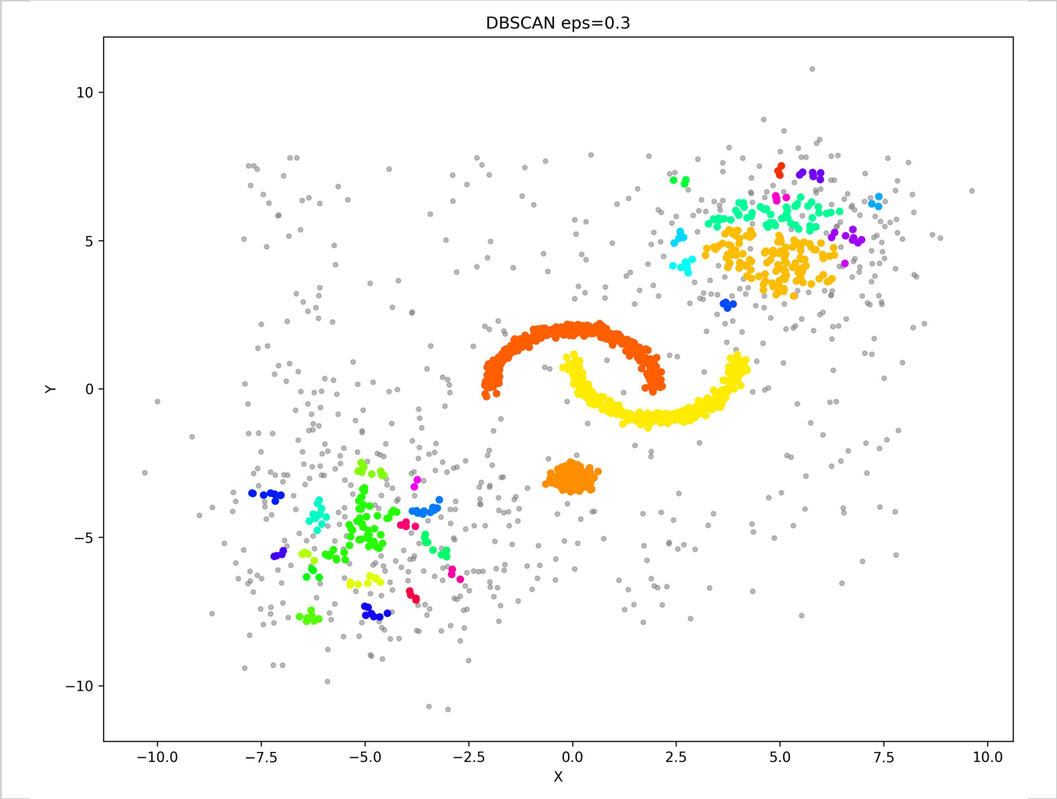
Local labels are only provisional until boundary components are merged.

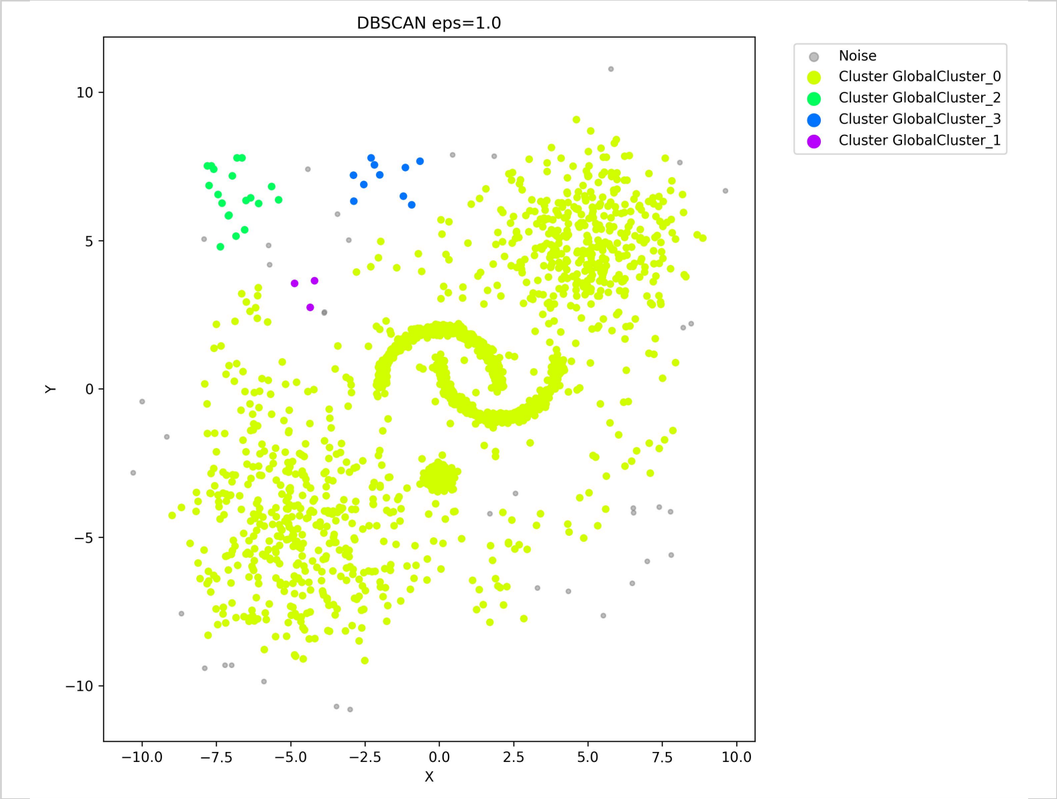
**Grid partition** → **Ghost points** → **Local DBSCAN** → **Union-Find merge**

**Distributed challenge: a neighbor may live in another partition, so labels must be reconciled globally.**

# Fixed eps creates opposite failure modes

The same algorithm fails differently depending on the radius.




**eps = 0.3 → sparse clusters become noise**

**eps = 1.0 → 96.5% points merge into one cluster**

# HDBSCAN replaces eps with density hierarchy

The key object is not a radius, but a stable tree of dense components.


local density scale

density-aware edge weights

keep essential connectivity

cut long edges over levels

select persistent clusters

**Single-machine view: global KNN, MRD graph, MST, hierarchy.**

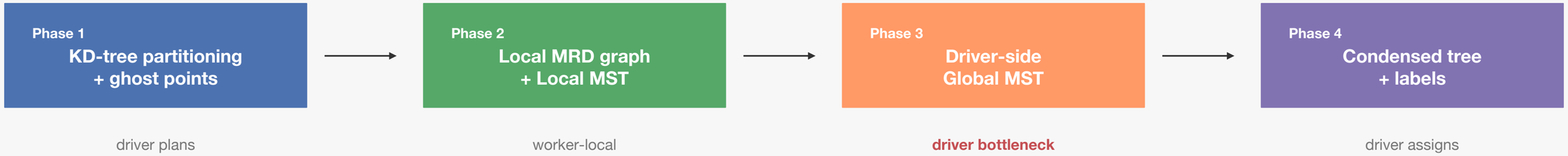
**Distributed view: local KNN/MRD + Local MST compression + driver merge.**

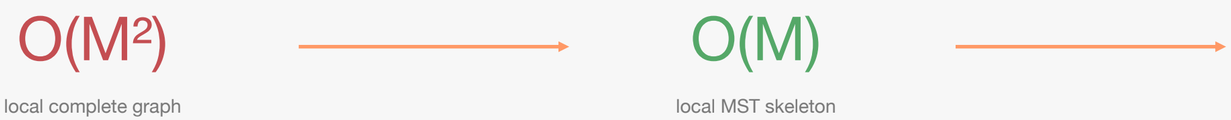
$$MRD(a,b) = \max(core(a), core(b), dist(a,b))$$

# Distributed HDBSCAN-inspired pipeline

Push local graph work to workers; merge compressed edges on the driver.

**Design principle: keep expensive graph construction local; move only compressed connectivity evidence.**



03

# Spark Implementation

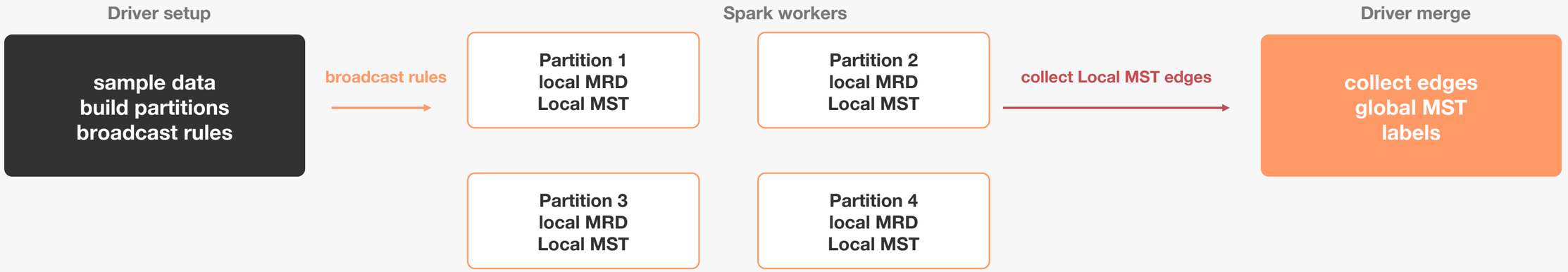
The implementation is a sequence of RDD transformations plus driver-side graph logic.

# System architecture

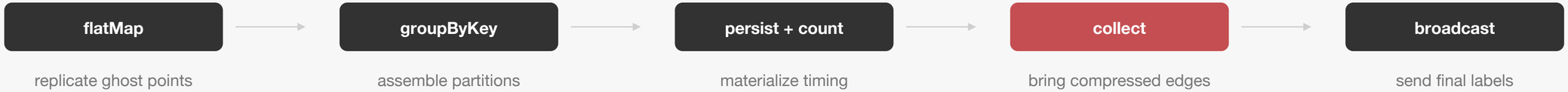
RDD pipeline with explicit driver / worker responsibilities.

## One driver coordinates; workers do partition-local graph work in parallel.

The only heavy movement is compressed connectivity evidence, not the full local MRD graph.




### RDD execution path

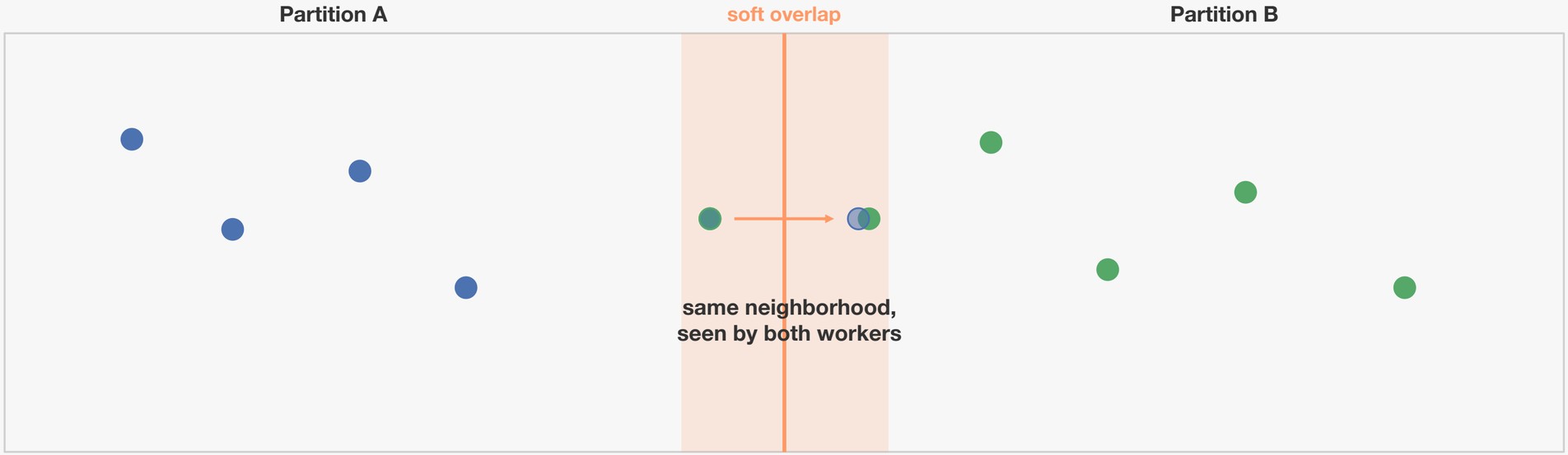


# Soft boundary with ghost points

Boundary replication protects cross-partition density connectivity.

## A hard partition boundary can split one density-connected neighborhood.

Ghost points create a soft overlap band so each worker can see near-boundary neighbors.




1 assign primary points

2 replicate boundary range

3 run local DBSCAN / MRD graph

DBSCAN uses eps; HDBSCAN-inspired uses max_dist as the overlap radius.

# Local MST compression

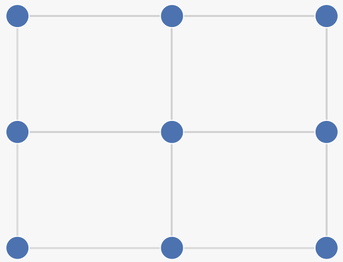
The local graph is dense; the tree keeps the essential skeleton.

**Each partition first builds a dense local MRD graph, then keeps only MST edges.**

This is the main communication-saving step before driver-side global merging.

Before

**local MRD graph**



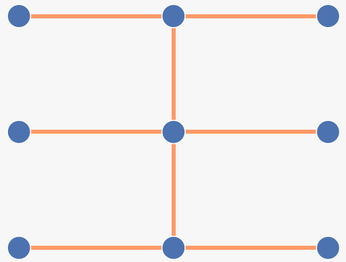
$O(M^2)$

candidate local edges

Kruskal
inside partition

After

**Local MST skeleton**



$M - 1$

local primary edges

**collect compressed edges only**

# Implementation map

Each code module owns one algorithmic responsibility.

| | |
|---|---|
| **core/partitioning.py** | Grid + KD-tree partitioners, ghost points |
| **dbscan/local_dbscan.py** | local distance matrix + BFS expansion |
| **dbscan/distributed.py** | partition → local DBSCAN → Union-Find merge |
| **hdbscan/local_graph.py** | core distance + MRD + Local MST |
| **hdbscan/distributed.py** | 4-phase HDBSCAN pipeline |
| **hdbscan/tree_hierarchy.py** | simplified condensation + stability labels |

**manual internals** **RDD-first** **phase timing** **WPS-editable shapes**

04

# Experiments + Discussion

The evidence tests both clustering quality and distributed scalability.

# Experimental setup

Four experiments, two types of evidence.

**E1** **Synthetic quality**
variable-density clustering

**E2** **Strong scaling**
cores: 1 / 2 / 4

**E3** **Data scalability**
N: 1k / 2k / 5k / 10k

**E4** **NYC Taxi**
real geographic density

## Visual quality

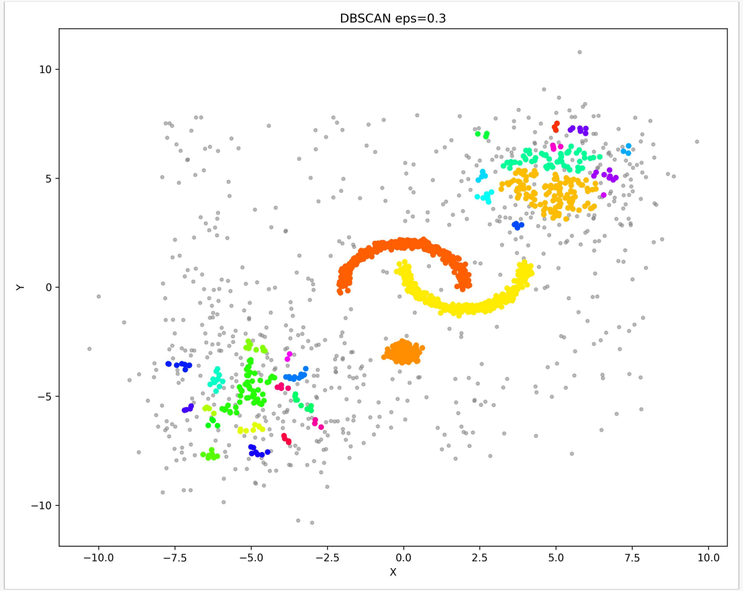
cluster shape · noise · over-merging

## System scalability

phase time · speedup · edge counts

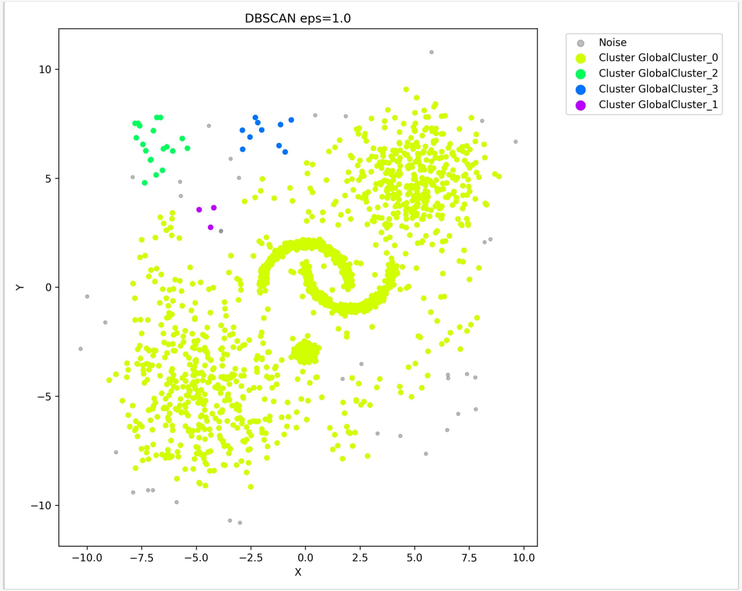
# Experiment 1: variable-density data

DBSCAN shows parameter sensitivity; HDBSCAN preserves more structure.



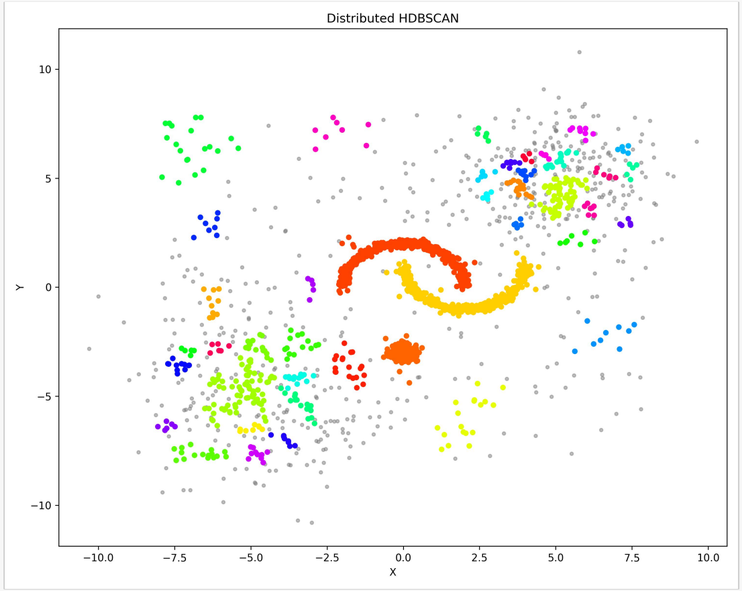

31.3% noise

**small eps**




96.5% in one cluster

**large eps**

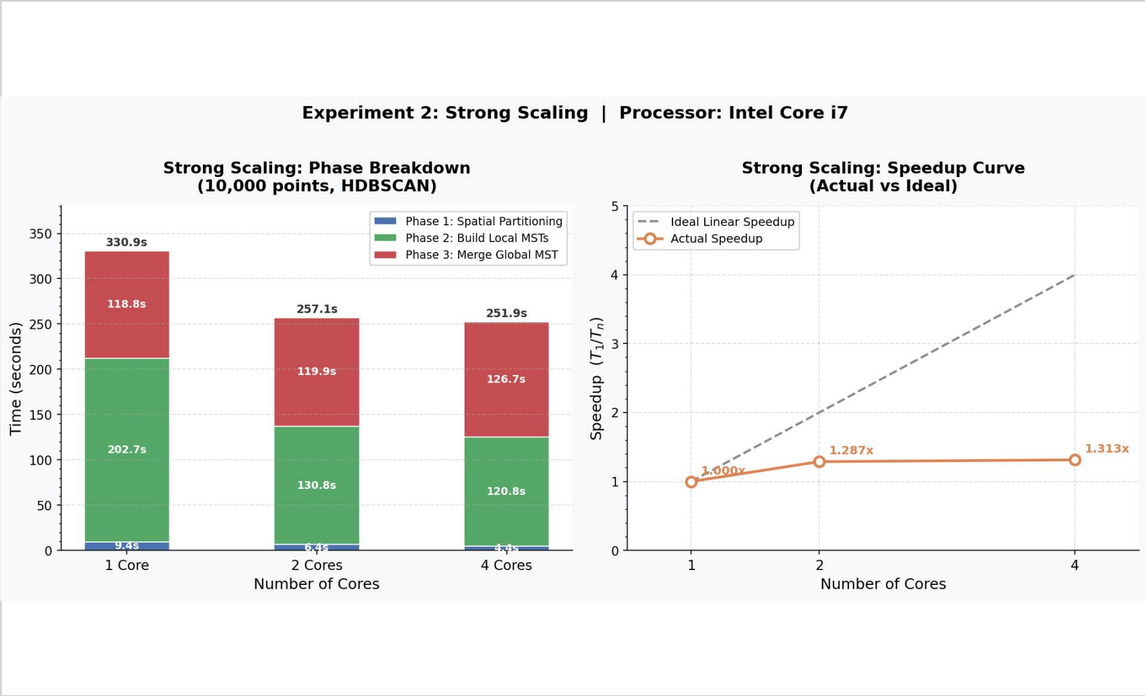


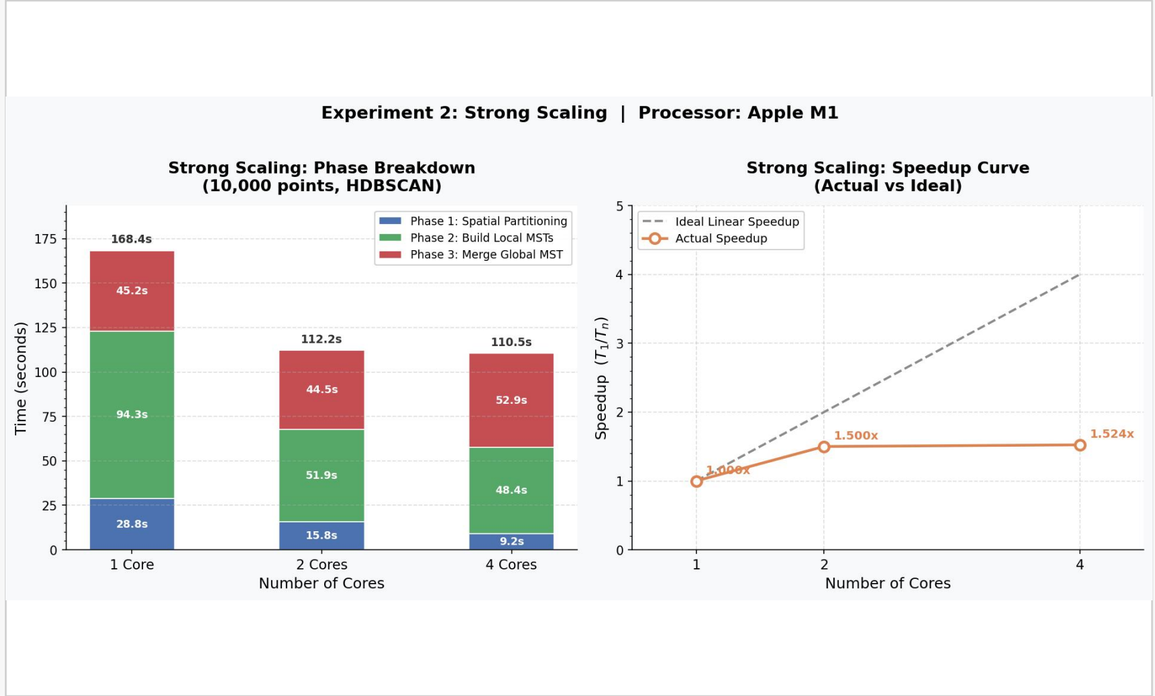

42 clusters · 24.1% noise

**hierarchy**

# Experiment 2: strong scaling

Parallel local work improves; driver merge does not.




**1.313×**

i7 speedup at 4 cores

**1.524×**

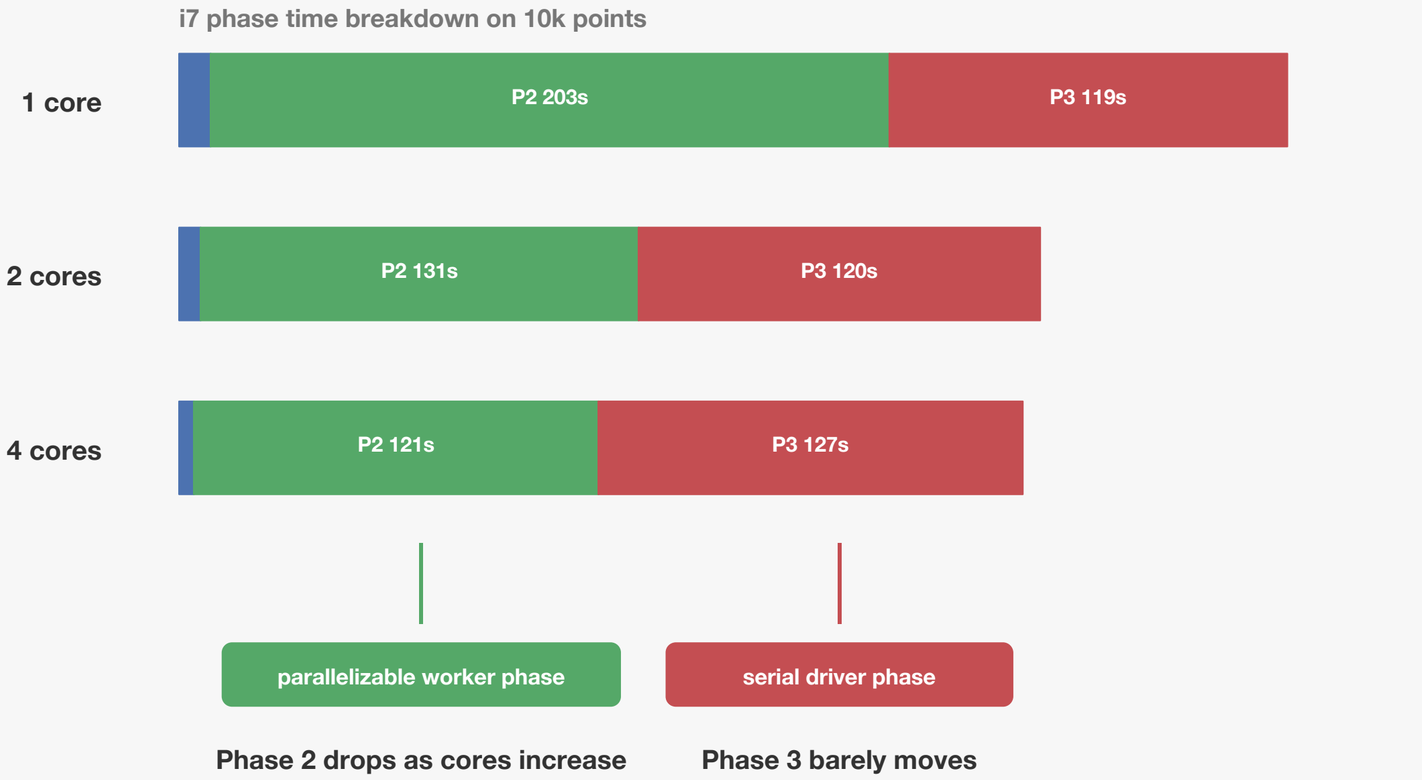
M1 speedup at 4 cores

**Amdahl bottleneck: Phase 3 Global MST**

# Why scaling flattens

The serial global merge becomes the floor under total runtime.

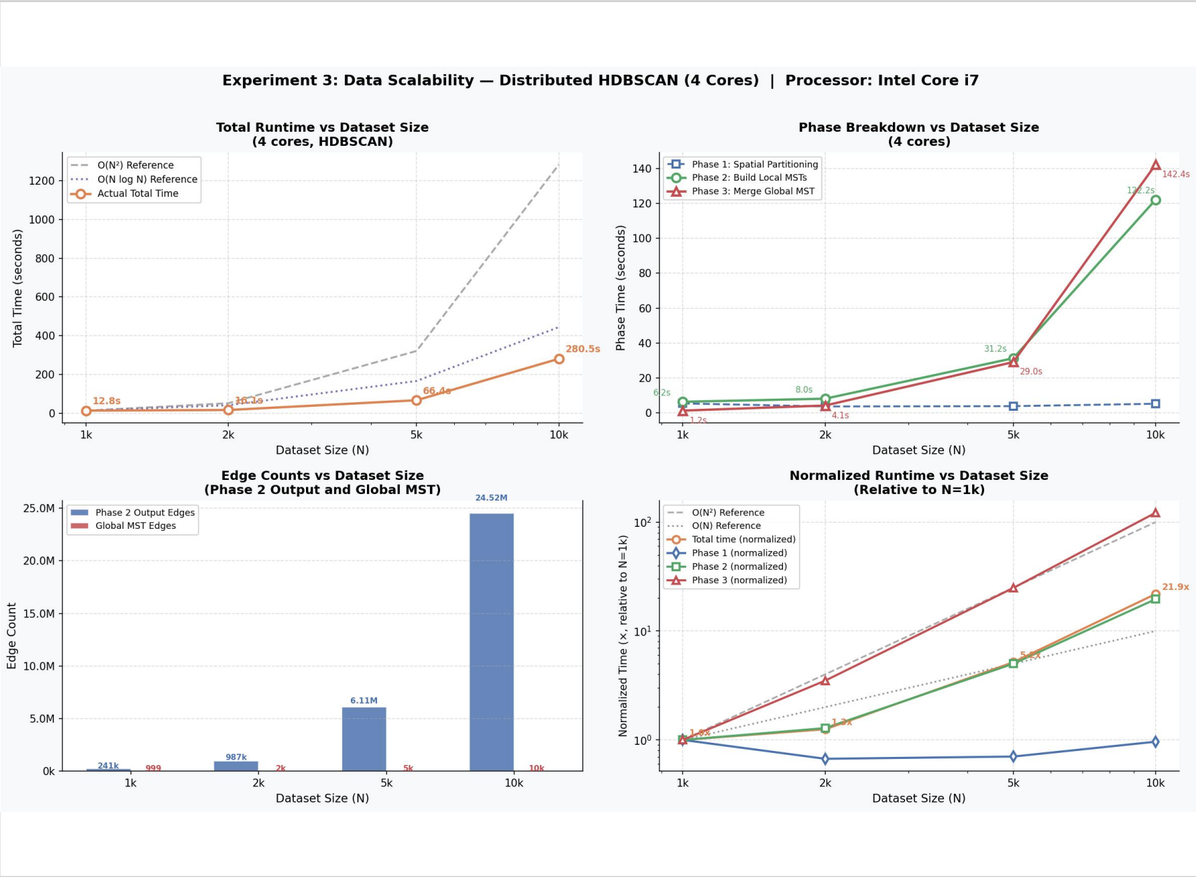
**Strong scaling improves worker-local work, but cannot remove the driver-side merge floor.**

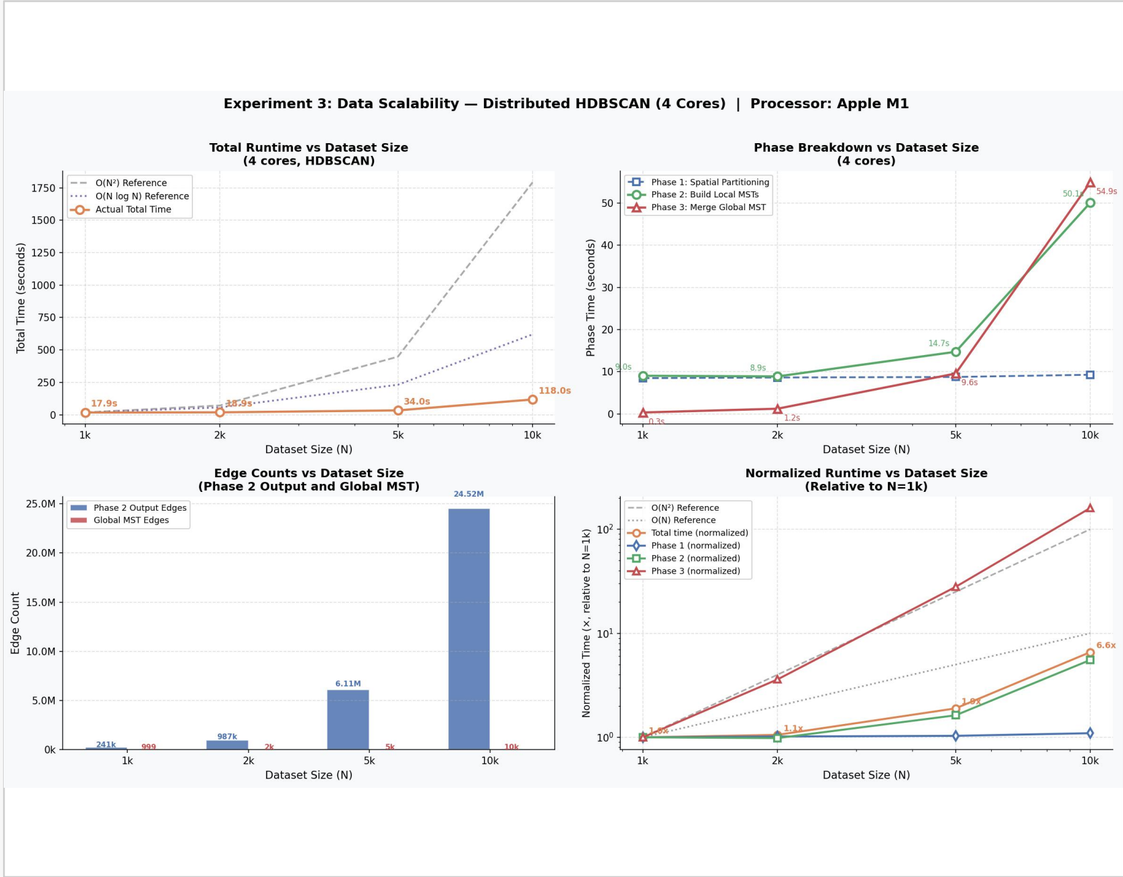



**Result: total speedup is capped even when local MST construction gets faster.**

# Experiment 3: data scalability

The final MST is linear, but candidate edges still grow fast.



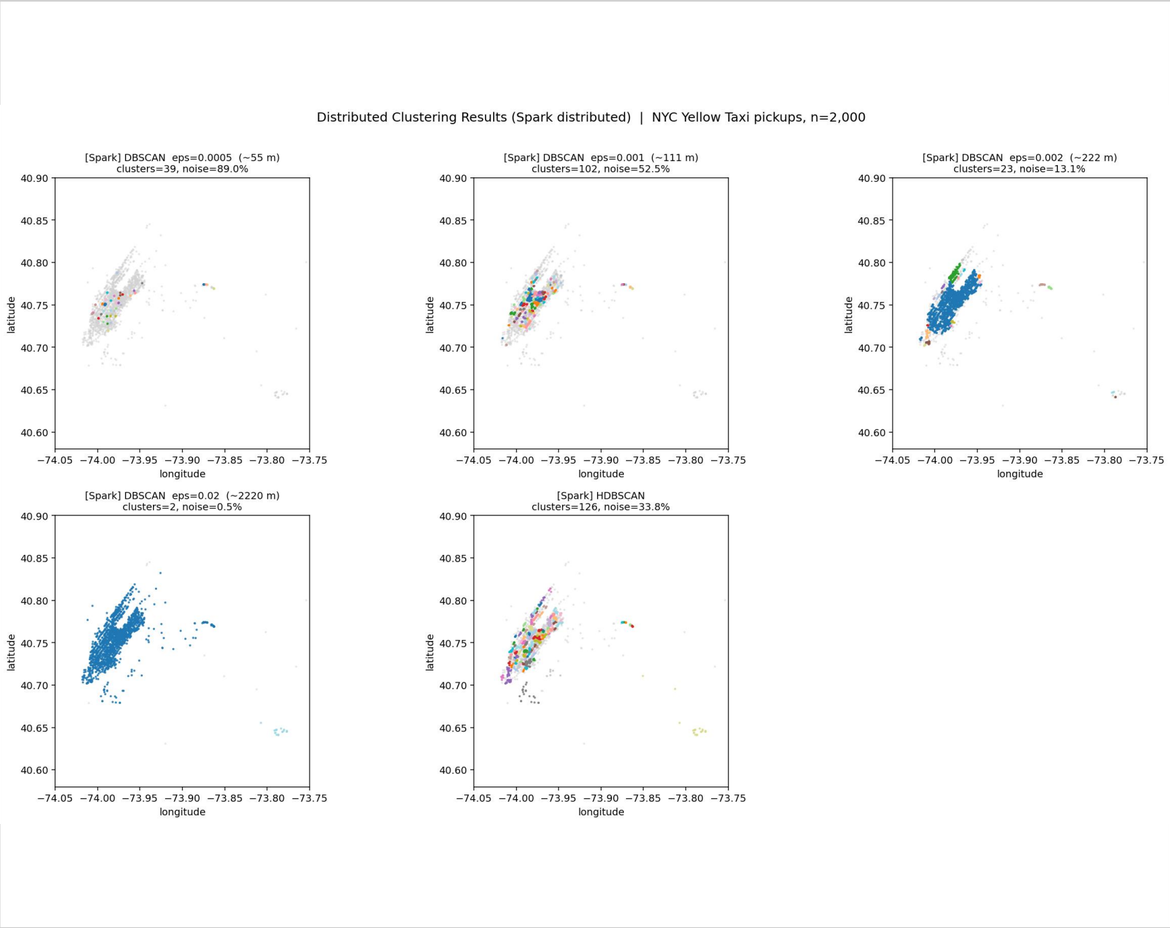

**10k complete graph: 49,995,000 possible edges**

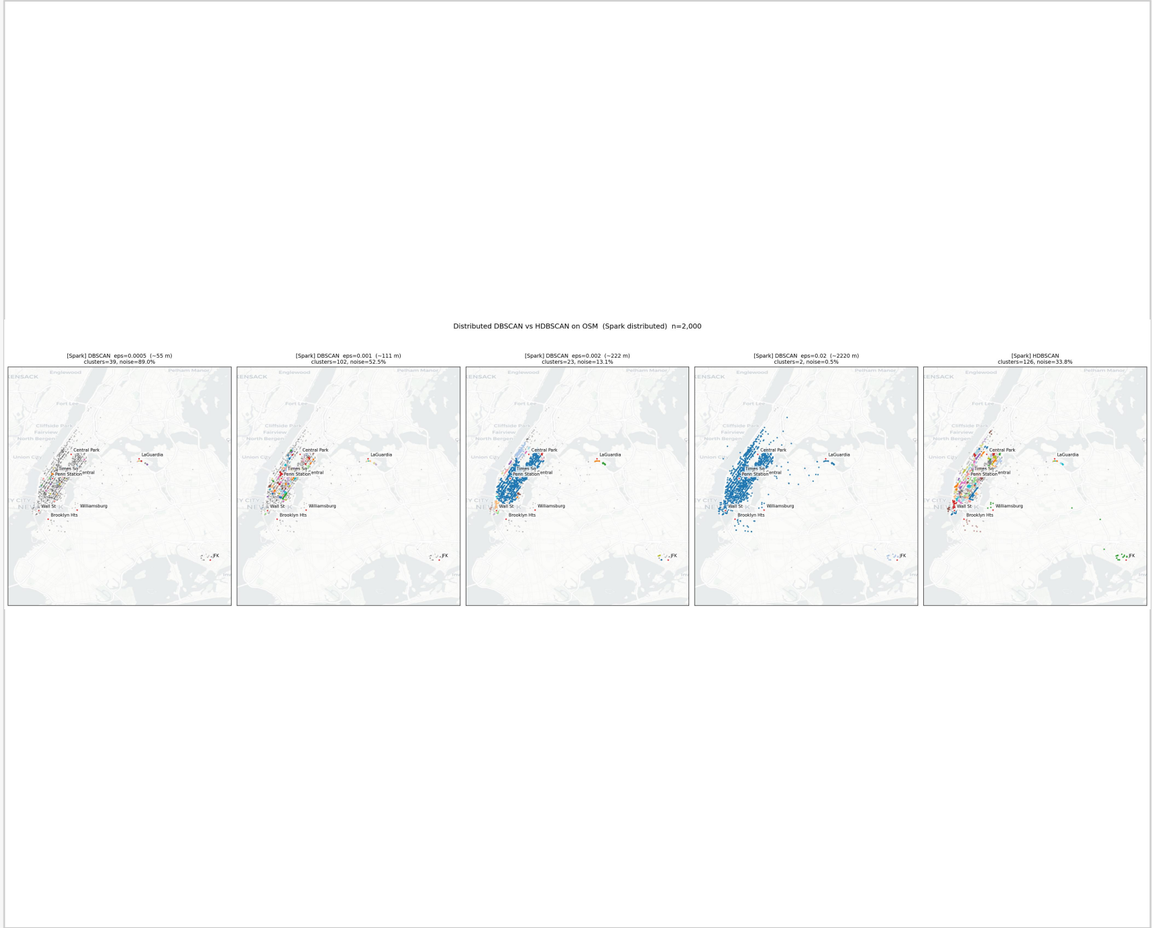
**Global MST: 9,999 edges**

**Phase 2 candidates: 24.52M**

# Experiment 4: NYC Taxi, n = 2,000

Real geography makes DBSCAN eps sensitivity visible.



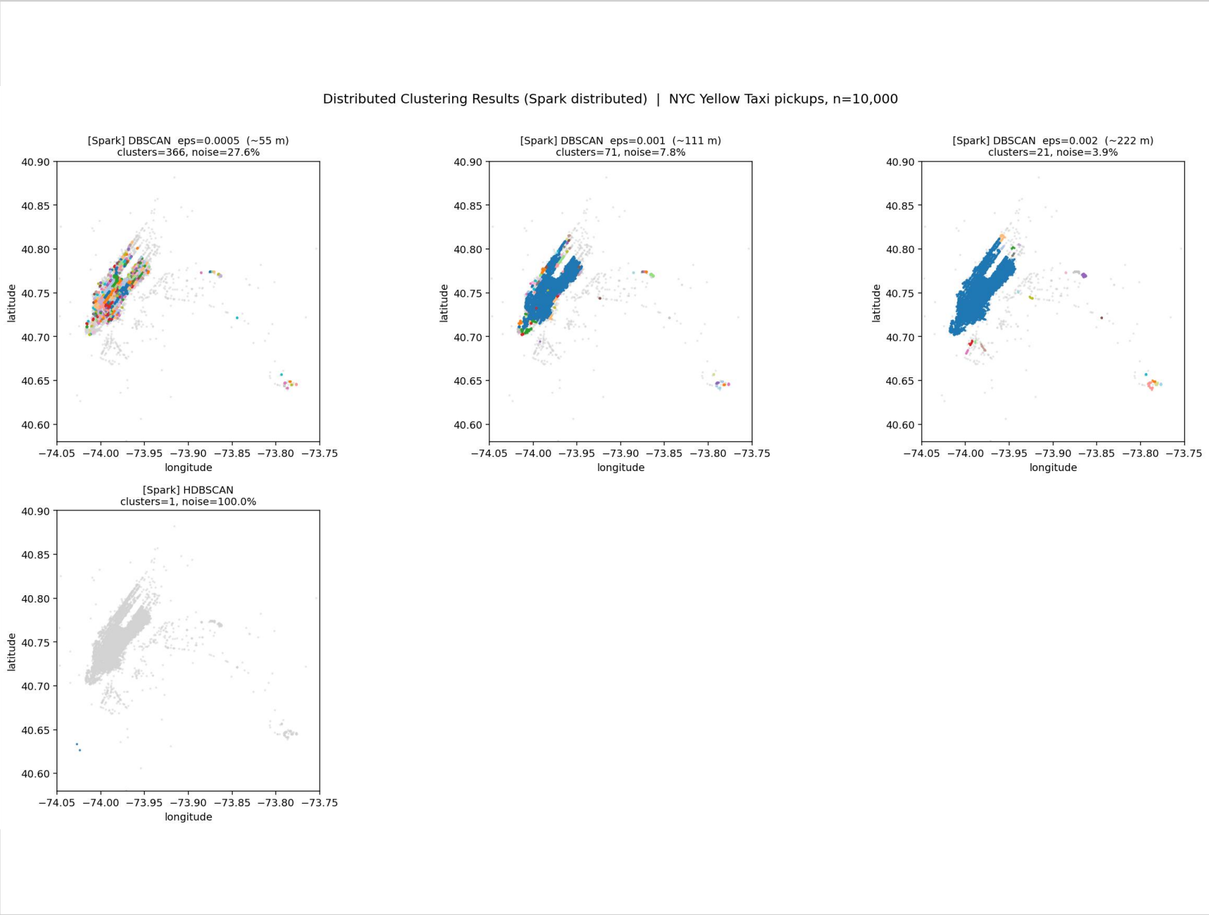

**eps 0.0005: 89.0% noise**

**eps 0.02: 98.3% in largest cluster**

**HDBSCAN: many local structures**

# Taxi 10,000 reveals the approximation limit

The candidate graph can become a forest, not one clean MST.

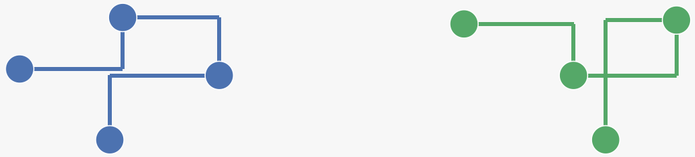



## Observed limitation

**HDBSCAN-inspired: 9,998 / 10,000 noise**

**This is a limitation of our approximation, not of HDBSCAN theory.**

### Candidate graph after compression



## No single-root tree

simplified hierarchy extraction expects one dominant connected structure

**forest → unstable labels / excessive noise**

# What the experiments say

The design works, but the bottlenecks are visible.

- **Algorithm** — HDBSCAN-style hierarchy handles variable density better than fixed eps.
- **Parallelism** — Local MRD + MST construction benefits from more cores.
- **Bottleneck** — Driver-side Global MST limits strong scaling.
- **Boundary cost** — max_dist controls connectivity but can inflate candidate edges.
- **Approximation** — Local KNN + forest cases limit real-data robustness.

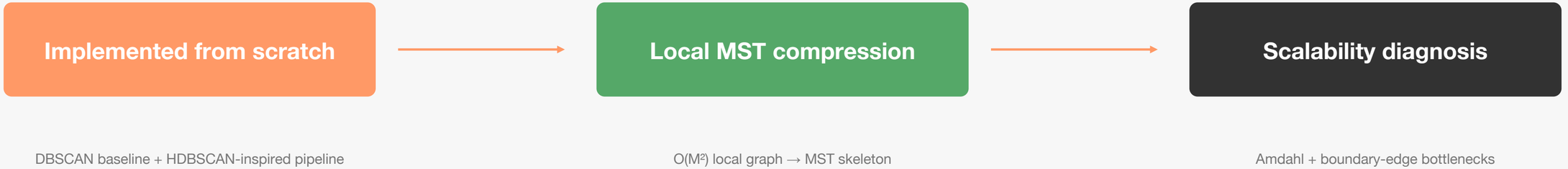
05

# Conclusion

The project turns density clustering into a distributed graph-compression pipeline.

# Final takeaway

A distributed HDBSCAN-style system is feasible when graph work is compressed early.




**Main contribution**

Mapping HDBSCAN’s MRD + MST + hierarchy logic into a Spark/MapReduce-style execution plan.

# Q&A

Distributed clustering · graph compression · Spark bottlenecks